

Support Vector Machines (SVMs)

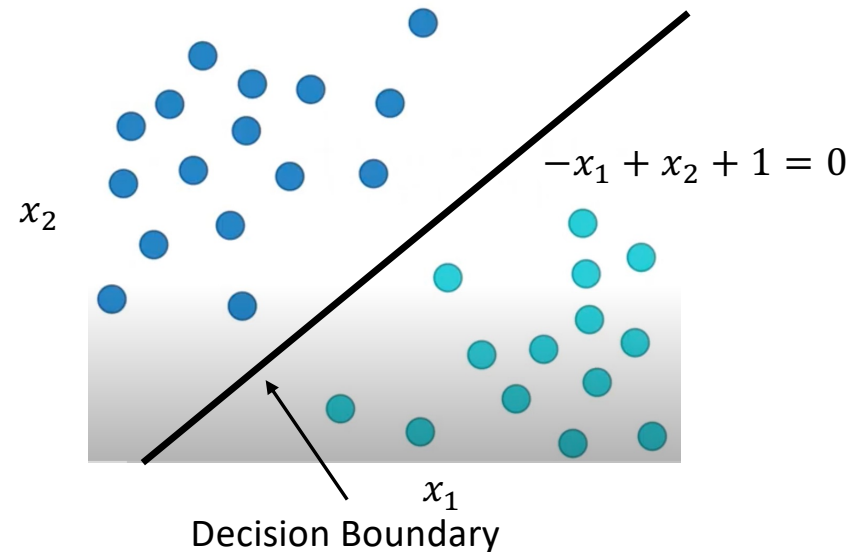
Jun Hong

Outline

- Basic concepts of SVMs
- Linear classification
- Non-linear classification
- Kernels

What is SVM?

- It's a classification algorithm like logistic regression
- Classification is about splitting data points into different classes
- We aim to fit a classification model to training examples in a dataset to get a classifier
- In *logistic regression*, after a model is trained, we have a decision boundary (a straight line in 2-d feature space) to split data points into two classes



In general, we have a decision boundary of the form:

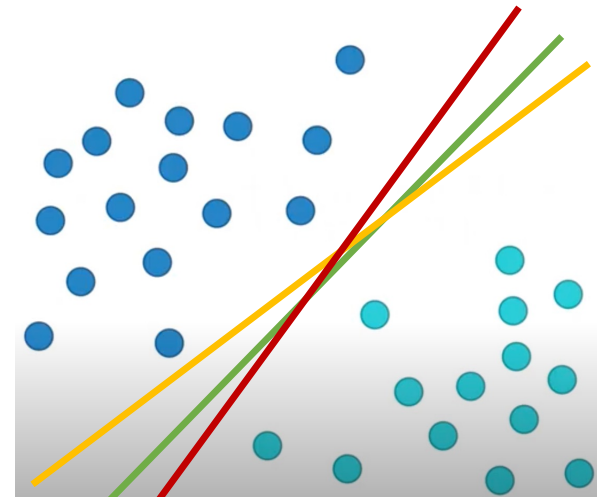
$$w_1x_1 + w_2x_2 + b = 0$$

We know that w_1 , w_2 and b are from the fitted model.

Note that w_1 , w_2 and b are θ_1 , θ_2 and θ_0 in a logistic model. ³

Classification is about splitting the data

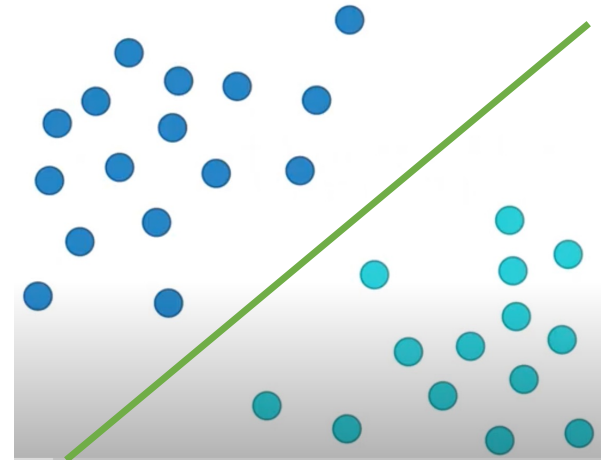
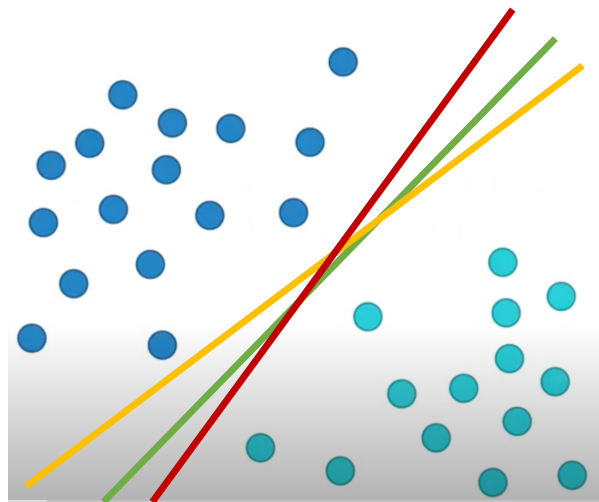
- We want to have a decision boundary to split two classes, as shown on the left, and we can have a straight line to separate the two classes as we do in logistic regression, as shown on the right



Different decision boundaries can be generated using a different set of values for the weights and bias.

Classification is about splitting the data

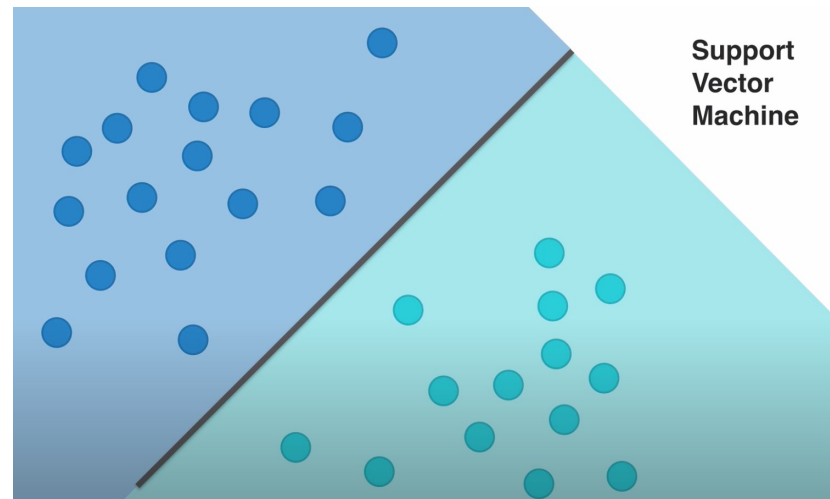
- However, the two classes can be separated by many different straight lines, as shown on the left
- So which line is the best? The one as shown on the right! But why?



Logistic regression can find a line to separate two classes, which can be any of these lines (if their weights and biases are all near the optimal), as shown on the left.

SVM – the Basic Idea

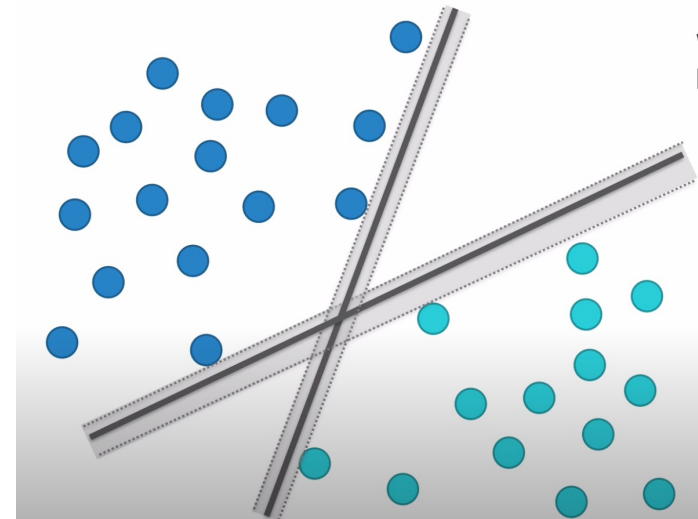
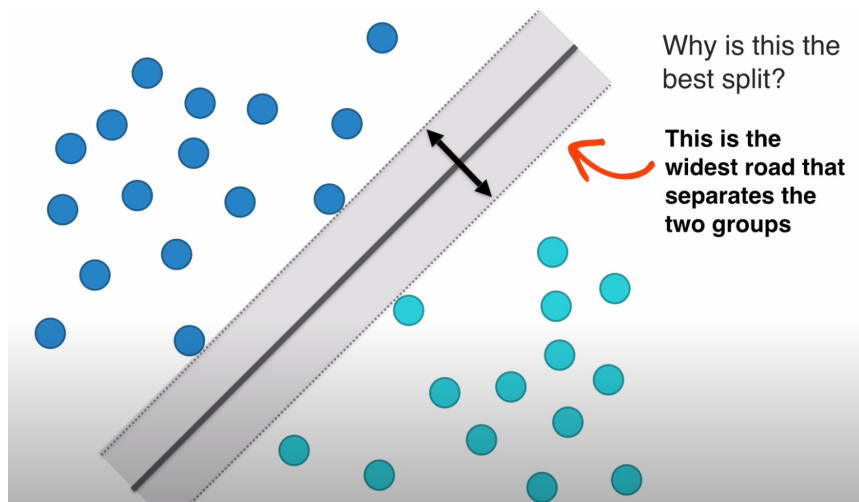
- This line as shown seems to be the best! But why? *Because it's the best to separate the two sections*
- But how do we know it's **the best**? This is what a Support Vector Machine (SVM) is about – *finding the best decision boundary, i.e. a threshold, line, plane or hyperplane to separate two classes*



An SVM tells us which decision boundary is the best to separate the two sections.

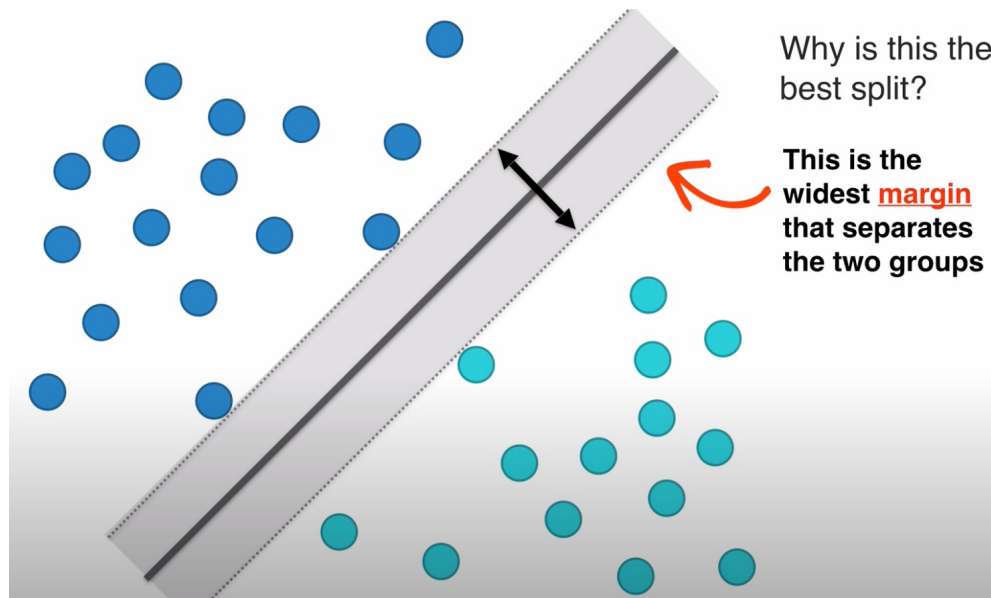
SVM – the Basic Idea

- Intuitively the line is the best to separate the two sections because if we add two lanes to both sides of the line we can build *the widest road*!
- The line in the middle of the road gives *the widest lanes on both side*; Otherwise, one lane is narrower so the traffic in that lane is prone to errors.



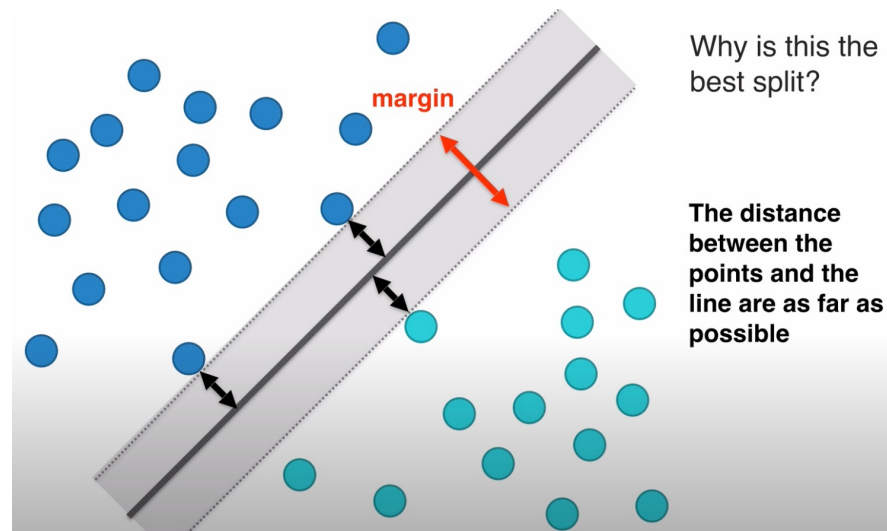
SVM – the Basic Idea

- Technically, the width of the road is called the *margin*. So this road has the widest margin compared to all the other roads.



SVM – the Basic Idea

- The margin (red arrows) is the maximal because the *distance* (black arrows, *half of the margin*) is the furthest to the data points at the edges (*the other data points do not matter*), compared to all the other roads.

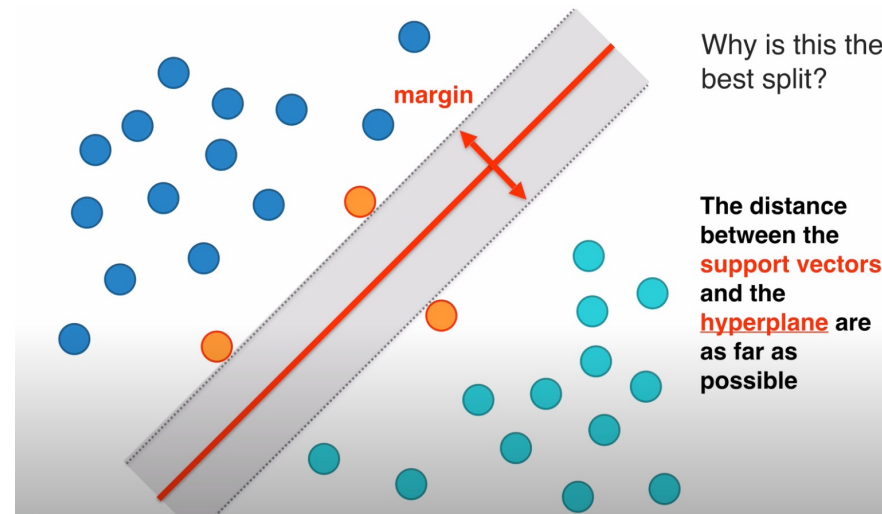


Distance is measured by Euclidean distance. The distance between a data point to a line is the shortest distance between the data point and the line, which is the line segment from the data point to the line perpendicular to the line.

SVM – the Basic Idea

- The red line in the middle is called *hyperplane* and those yellow data points at the edges of the road are called *support vectors*.
- The goal of SVM is to find the hyperplane that has the *maximal margin* (i.e. to have *furthest distance* to the support vectors).

The edges of the road are parallel to the line in the middle, i.e. have the same slope.



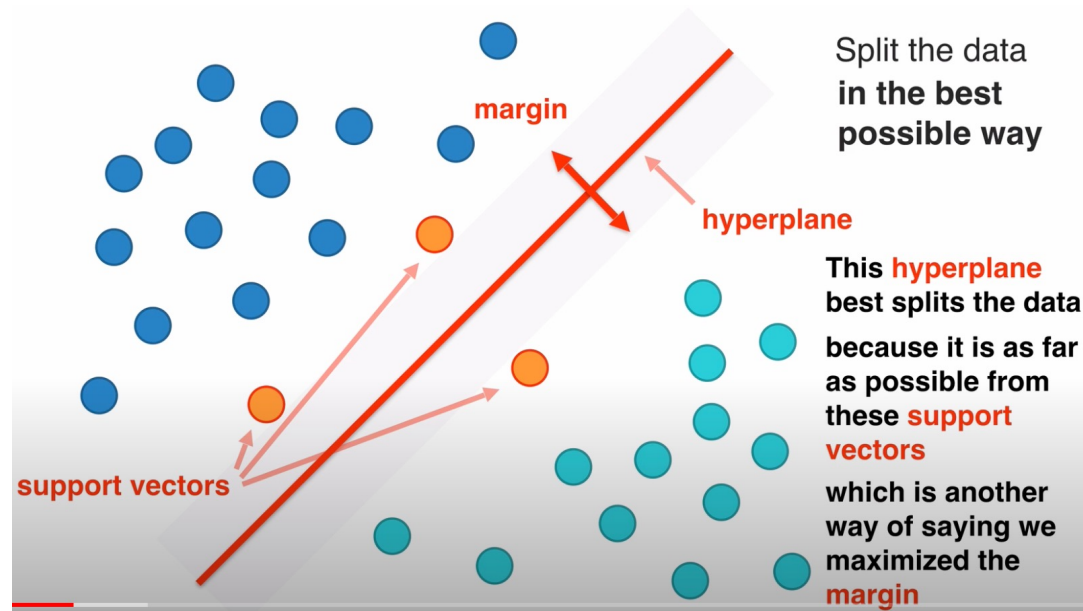
Why is this the best split?

If we have the widest lanes on both side of a road, we have the widest road.

The distance between the **support vectors** and the **hyperplane** are as far as possible

Only the data points at the edge of the road matter as far as the widest road is concerned – the other data points do not matter.

SVM – Summary of the Basic Idea



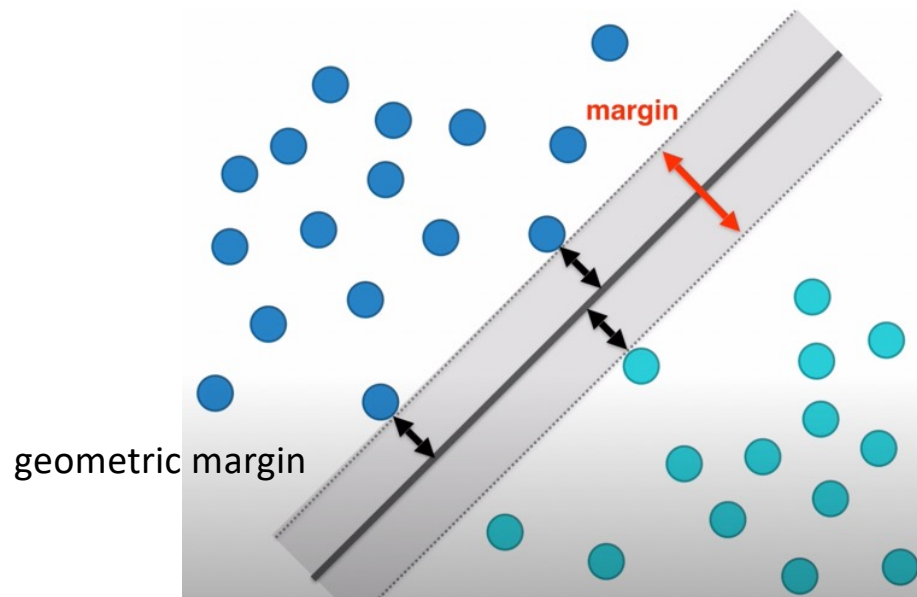
How do we find a hyperplane that has the maximal margin? This is a *constrained optimisation* problem, i.e. the hyperplane maximising the margin while keeping the support vectors off the margin.

Optimal/Maximal/Largest Margin Classifiers

- We need to fit the model $wx + b = 0$ (i.e. choose w, b) with the training examples, to maximise g (the *geometric margin*), the *Euclidean distance* between the hyperplane and the worst-case training examples (*support vectors*) on both sides of the hyperplane
 - When the hyperplane is halfway between the support vectors, the margin is as large as it can be, which is the same on both side.
 - $margin = 2 \times \text{geometric margin of support vectors}$
- The maximal margin allows any new example that is closer to the group of training examples on one side of the hyperplane falls on that side of the hyperplane; Otherwise, such a new example may fall on the other side of the hyperplane

Margin versus Geometric Margin

margin = 2 × geometric margin of support vectors



Geometric margin is the *Euclidean distance* between the support vectors and the decision boundary, which is *the shortest distance* between any data point and the decision boundary

The shortest distance between a data point and the line, is the line segment from the data point to the line perpendicular to the line.

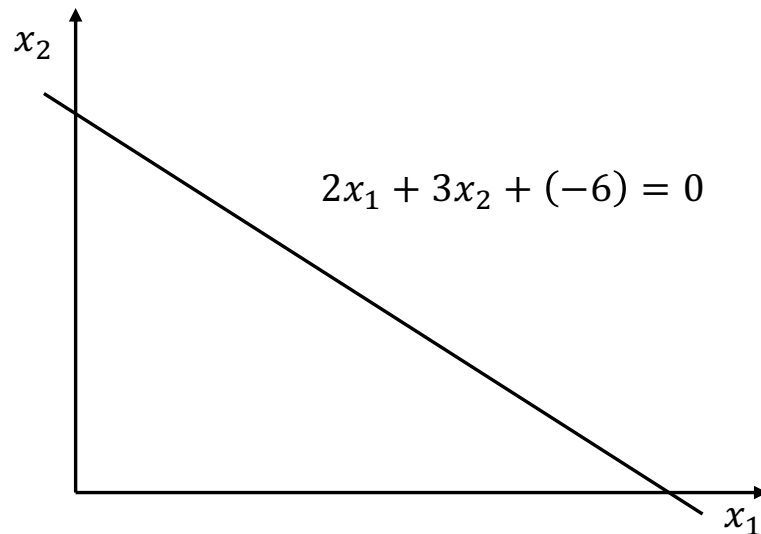
The Optimisation Problem – the Algorithm

- To find a pair of w and b that maximise the *geometric margin* (*maximal geometric margin*)
 - For each possible pair of values for w and b , we get the *geometric margin* that is the *geometric distance* from the decision boundary to *the nearest example* (the worst/shortest/minimal geometric distance from every training example in the dataset to the decision boundary)
 - We can calculate each possible *geometric distance* from the decision boundary to each data point and choose the closest/smallest one as the *geometric margin*
 - We can then compare all possible geometric margins and choose the maximal one

Each pair of w and b has a geometric margin. The pair that has the maximal geometric margin is the one we are looking for. Note that w may consist of a set of values (weights) and b is the bias.

We can consider the maximal geometric margin as a cost/loss function with an iterative process to maximize this function.

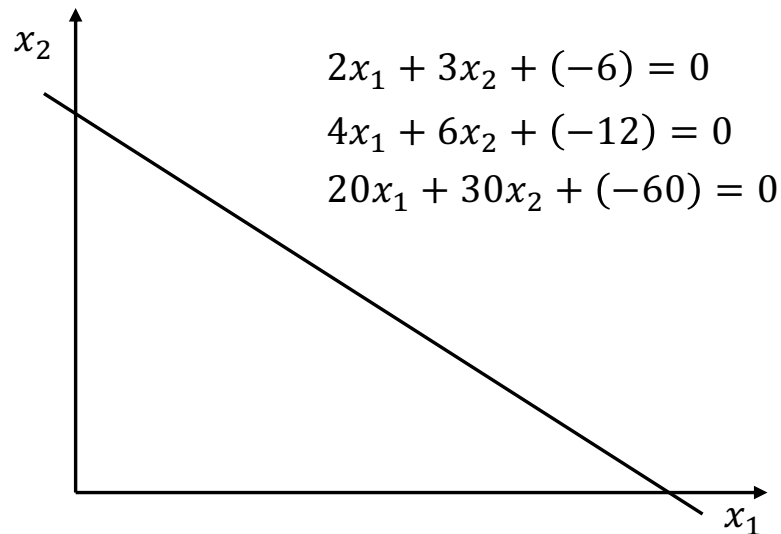
How to Find a Possible Line and Its Geometric Margin?



- As we can see, a linear equation, e.g. $2x_1 + 3x_2 + (-6) = 0$ represents a straight line in a 2-d space
- We can have a generalised equation, $w_1x_1 + w_2x_2 + b = 0$ to represent any straight line in the 2-d space, depending on the values of the *weights*, i.e., w_1 and w_2 , and the value of the *bias*, i.e., b
- We need to find a possible set of values to satisfy *the constraint that we have the geometric margin*, i.e. the distance from the line to the support vectors (those data points on the *both edges of the road*)
- How to find these values, i.e. how to find a possible line that has the *geometric margin*, g ?

Try to draw another line $2x_1 + 3x_2 = 0$ to show the effect of the bias.

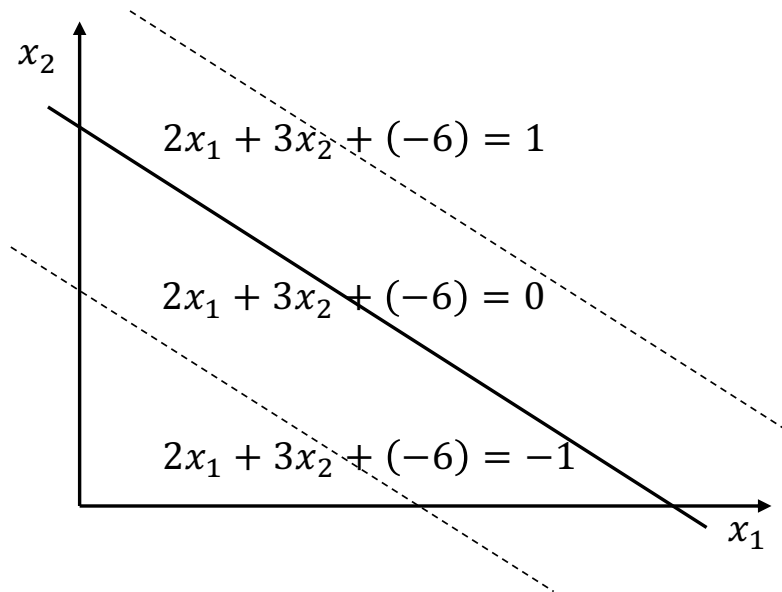
How to Find a Possible Line and Its Geometric Margin?



Why does the line not move?

- As we can multiply a constant to the equation to get another equation that represents the same line
- All three equations represent the same line (*if one side of the equation is 0*)
- We can multiply a factor/constant to the generalised equation, $w_1x_1 + w_2x_2 + b = 0$ to get another equation that represents the same line
- We can multiply a factor (i.e. a number) to the existing weights and bias to a new set of weights and bias to represent the same line

How to Find the Geometric Margin?

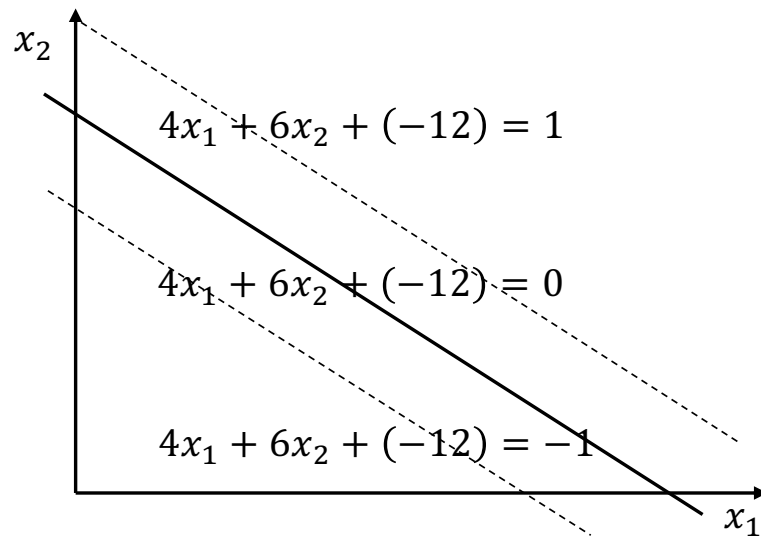


- As we can see the edges of the road have the same weights and bias as the line in the middle
- But one edge has 1 on the right side while another edge has -1 on the right side
- *The most important property* is that we can multiply a constant to the left sides of the three equations but the line in the middle stays in the same place while the margin between two new edges changes

Why do the three lines have the same slope?

Why do two edges move?

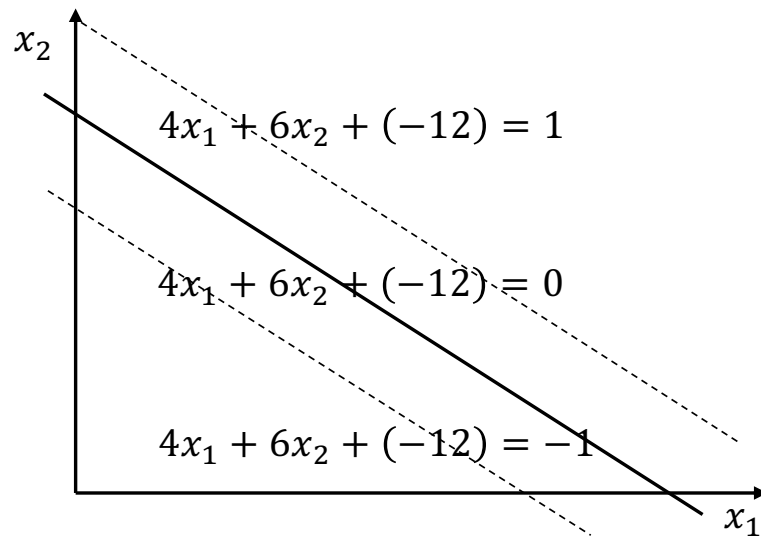
How to Find the Geometric Margin?



- As we can see if we multiply a constant 2 to the left sides of three equations, the line in the middle stays the same while the margin between two new edges gets narrower
- We can change the weights and bias in the three equations by multiplying a constant to the left sides to change the margin while keeping the same line in the middle
- The benefit of doing that is we can make the margin as wide as possible until the support vectors are at both edges
- We will then have found the geometric margin

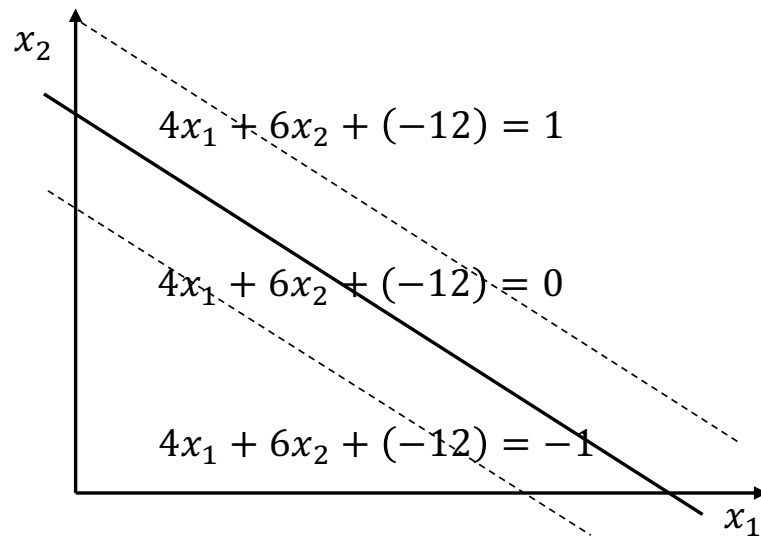
Why do the two edges get closer to the middle?

How to Find the Geometric Margin?



- We can start with a set of weights and bias for the three lines that separate the two groups of data points, and multiply an expanding factor very close to 1, say, 0.99 with two dotted lines very close to each other;
- We then gradually reduce a tiny bit for the expanding factor so that the dotted lines become a little bit further apart;
- We repeatedly do this until the two dotted lines are on the data points on the edges of the two groups. Finally, the distance between these data points on the edges is the furthest.

How to Find the Decision Boundary with Maximal Margin?



- In SVM, the decision boundary consists of three equations:
$$w_1x_1 + w_2x_2 + b = 1$$
$$w_1x_1 + w_2x_2 + b = 0$$
$$w_1x_1 + w_2x_2 + b = -1$$
- Apart from changing the margin, we can also change the slope of the lines by making either w_1 or w_2 smaller, we can make them more stretched along either x_1 or x_2 ;
- We can also change the bias, b ;
- We repeatedly do this with a different set of w_1 , w_2 and b , we can get different lines and their geometric margins;
- We can choose the one with the largest geometric margin as the decision boundary and its geometric margin as the maximal margin.

In mathematics and sci-kit learn, w_1 and w_2 are called *coefficients*, b is called *intercept*. Once a model is fitted, you can use `model.coef_` and `model.intercept_` to get coefficients and the intercept. We can also use `model.support_vectors_` to get all support vectors.

Fit an SVM Model

- The model is represented as the decision boundary consisting of the three equations with the values for w_1 , w_2 , b and c (a *hyperparameter*, discussed later) to maximise the geometric margin between two dotted lines that are on the support vectors
 - By multiplying the left sides of the equations simultaneously, we can make the lines either closer or further apart - the lines are furthest apart if the line segment from a support vector and the middle line is perpendicular to the middle line and the two dotted lines;
 - By changing the value of either w_1 or w_2 (i.e. the slope of the lines), and b (i.e. the distances of the lines from the origin of the coordinates).

Maximal (Hard) Margin Hyperplanes

- When we have a 3-d feature space, the maximal margin classifier is a 2-d plane that separates two 3-d spaces, as follows:

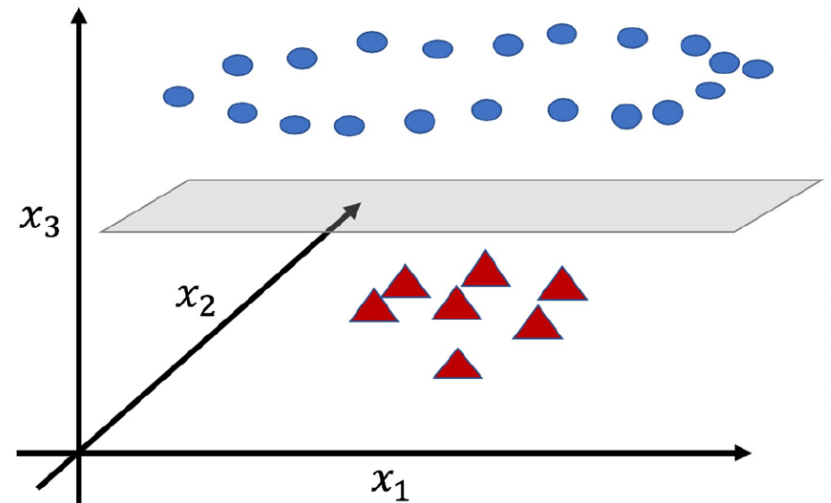
$$w_1x_1 + w_2x_2 + w_3x_3 + b = 0$$

- Accordingly, the maximal margin is between

$$w_1x_1 + w_2x_2 + w_3x_3 + b = 1$$

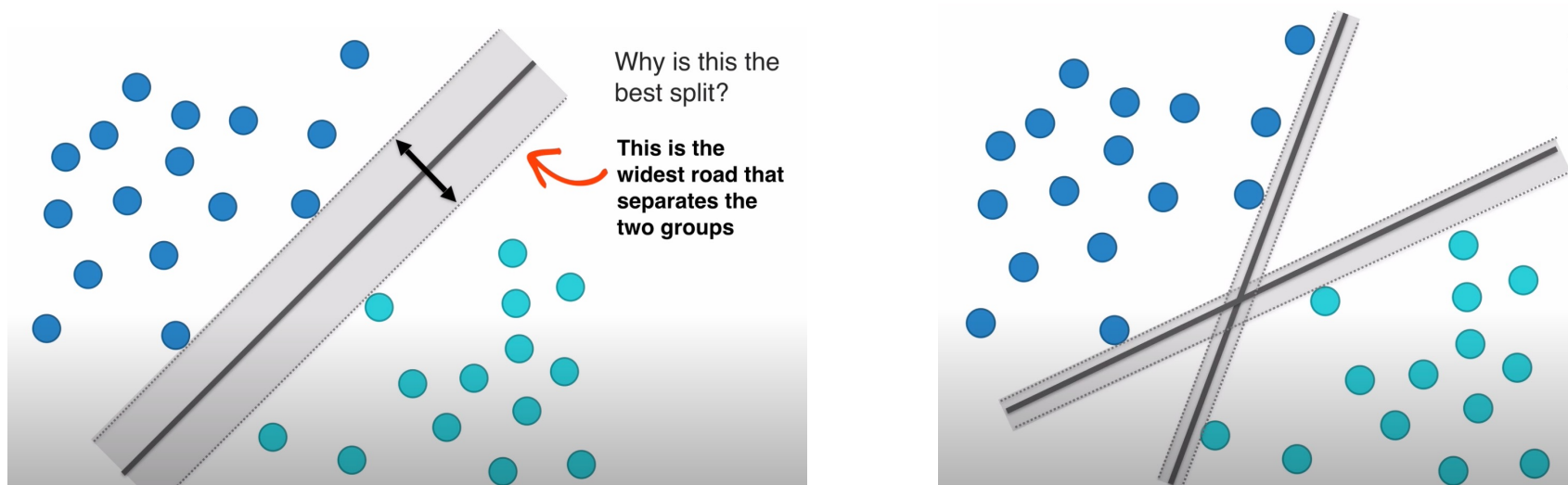
$$w_1x_1 + w_2x_2 + w_3x_3 + b = -1$$

- When we have a n -d feature space, the maximal margin classifier is a $(n-1)$ -d hyperplane that separates two n -d spaces that we cannot visualise



Wider Margin is better in some cases only

- When we have two hyperplanes that **clearly** separate the two classes, the one with the wider margin is always better
- *Margin error* – the reverse of margin, i.e. the wider the margin, the smaller the margin error, e.g. the road on the left is better

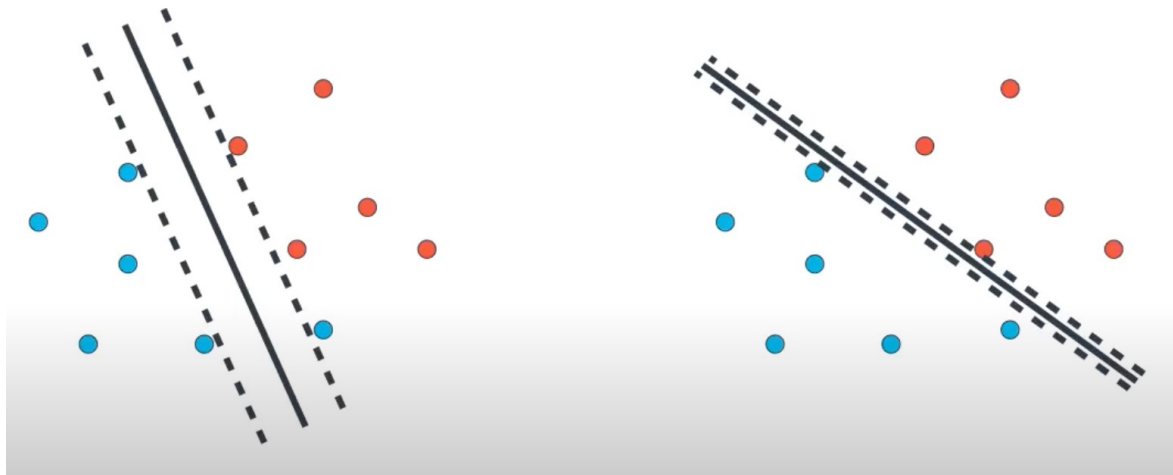


The goal is to minimize the margin error to fit the model to the training dataset.

Wider Margin may not be always better

- What happens if the hyperplane with the narrower margin clearly separates the two classes **while the hyperplane with the wider margin misclassifies some data points?**
- Which hyperplane is better? The answer is that it's up to us to decide

Lower margin error
but higher classification
error



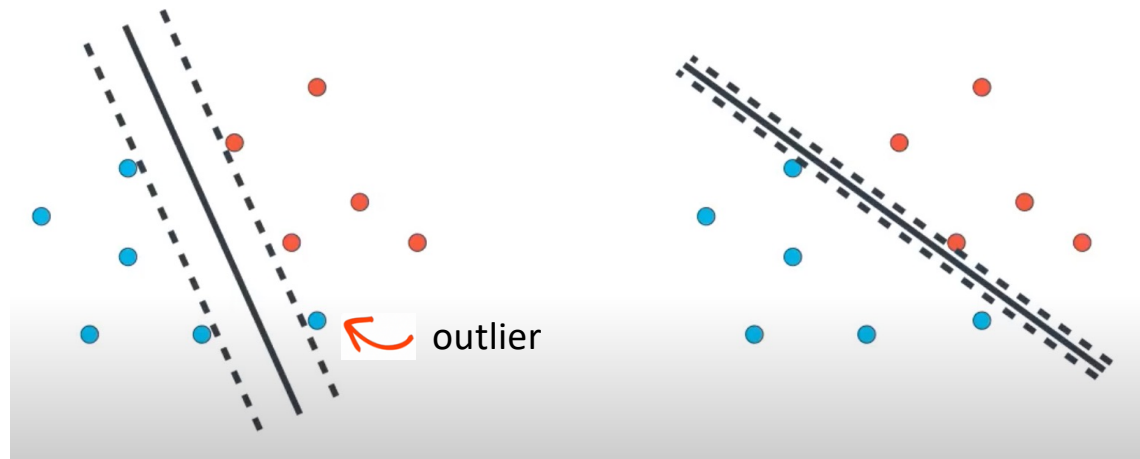
Higher margin error
but lower classification
error

But does this overfit? What
about its variance?

Soft Margin Classifiers

- The maximal margin classifiers *are super sensitive to outliers* (i.e., noise) in the training data
 - To make the hyperplane that is *not super sensitive to outliers* we must allow *misclassifications*
 - When we allow misclassifications, the margin is called the *soft margin* and the classifier is called the *soft margin classifier*

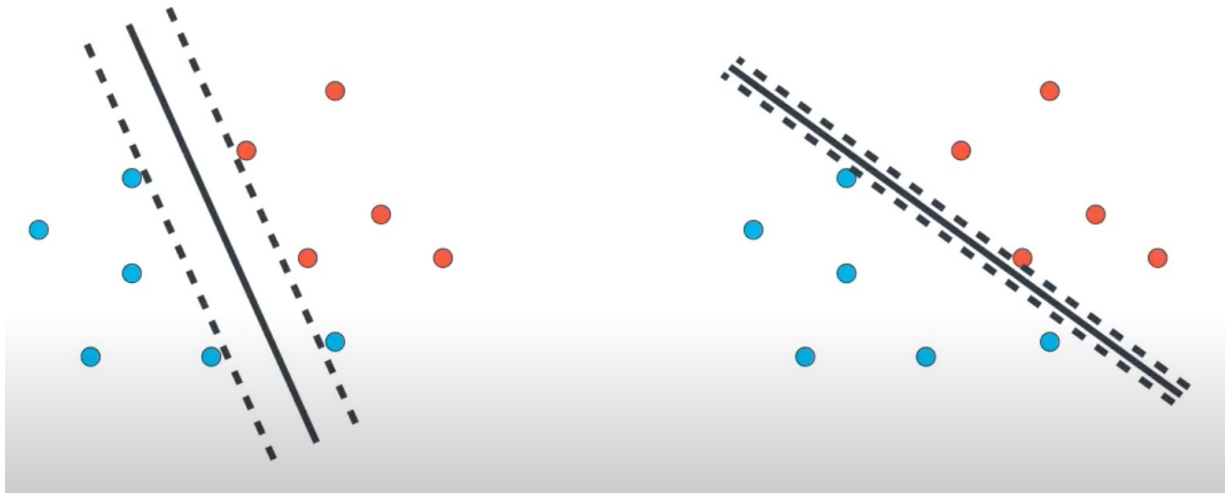
Outliers are against model generalization because they are not representatives of a class.



Classification Error and Margin Error

- But unlike the maximal (hard) margin that is unique, there are many possible soft margins, how do we know which one is better?
- We need to take into account two types of error in a balanced way, *classification error* and *margin error*, i.e. *bias/variance trade-off*
 - Classification error measures how well the hyperplane classifies the data points while margin error reflects how wide the margin is

High bias/underfitting
Low variance



Low bias/overfitting
High variance

Bias means the *subjectivity* to the data while variance means the *sensitivity* to variations.

End of Week 5

The C Parameter

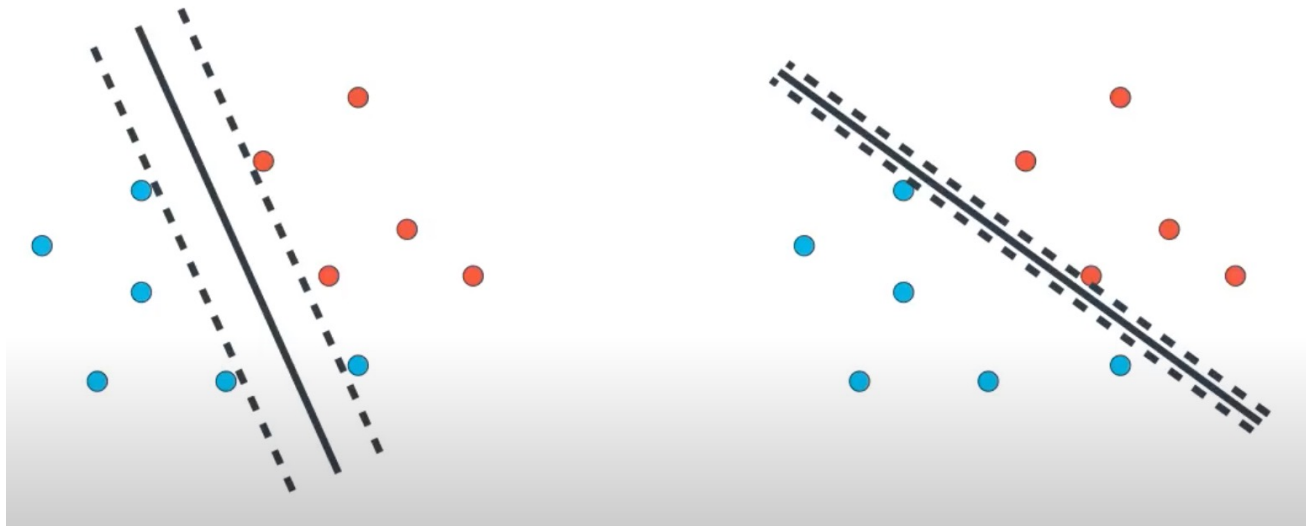
Given a set of candidate values for C , we can fit the model to the training dataset to get a set of different trained models each with a specific value of C . We need to validate these models with their value of C on the validation dataset to the best model with a particular value of C , a step called hyperparameter tuning.

- SVM allows us to assign a value for a *hyperparameter*, C , to give a weight to classification error:

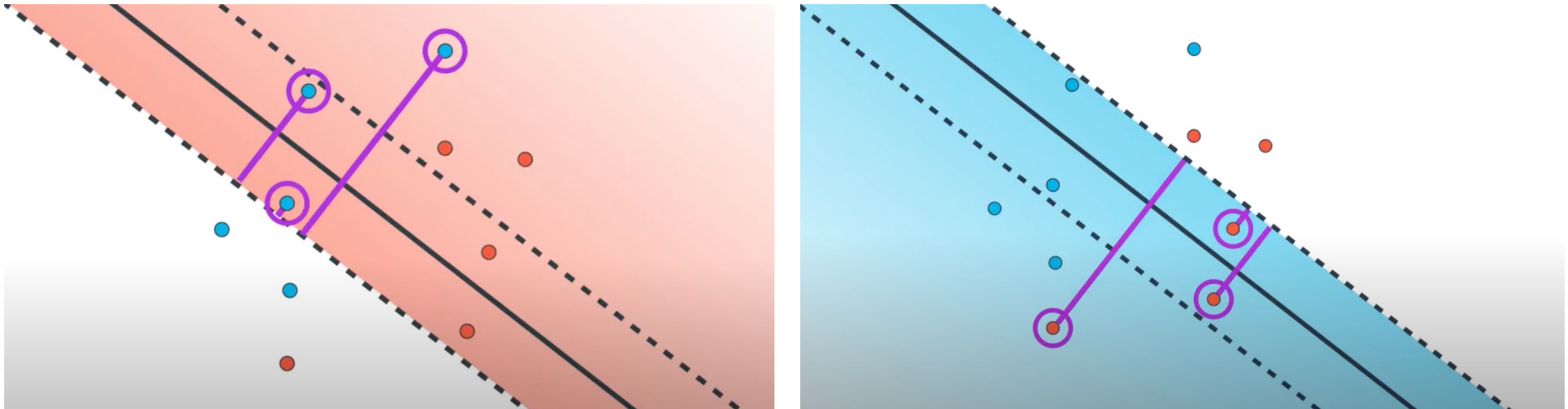
$$SVM\ Error = C \times Classification\ Error + Margin\ Error$$

- The larger C , the more important the classification error is; the smaller C , the less important it is – it's up to us to decide on the best value of C

For example, two different values for C , e.g. 0.1, 100, will lead to two different trained model fitted to the training dataset.



How to Calculate Classification Error



Classification Error = Red Classification Error + Blue Classification Error

The geometric/shortest distance (length of the line perpendicular to the margin line) between each misclassified data points and the correct side of the road/margin is calculated and summed.

How to Calculate Margin Error

The Euclidean distance d from any data point $x = (x_1, x_2)$ to the decision boundary is:

$$d = \frac{|w_1x_1 + w_2x_2 + b|}{\sqrt{(w_1)^2 + (w_2)^2}}$$

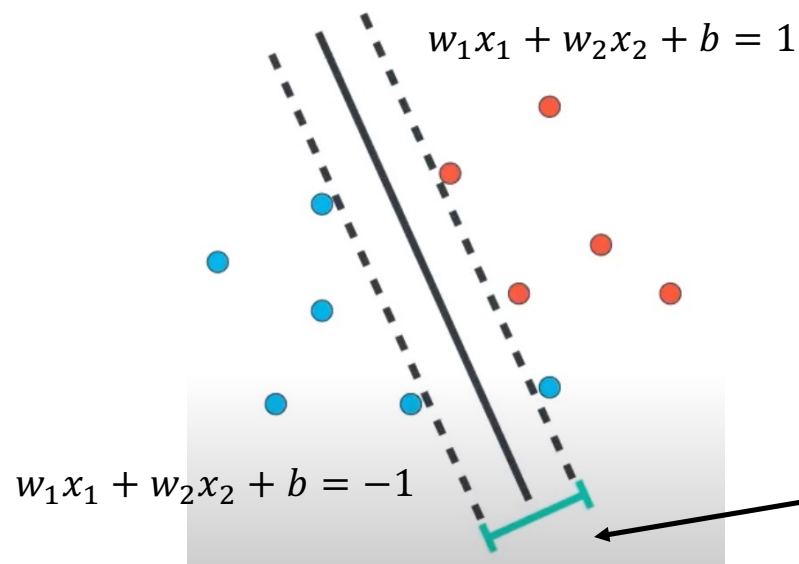
The geometric margin g from support vectors to the decision boundary is:

$$g = \frac{1}{\sqrt{(w_1)^2 + (w_2)^2}}$$

$$\text{Margin} = 2 \times g = \frac{2}{\sqrt{(w_1)^2 + (w_2)^2}}$$

Margin error is *inversely proportional* to margin:

$$\text{Margin Error} = \frac{1}{2} ((w_1)^2 + (w_2)^2)$$



Recap – What We have Learned So Far

- Linear regression and gradient descent

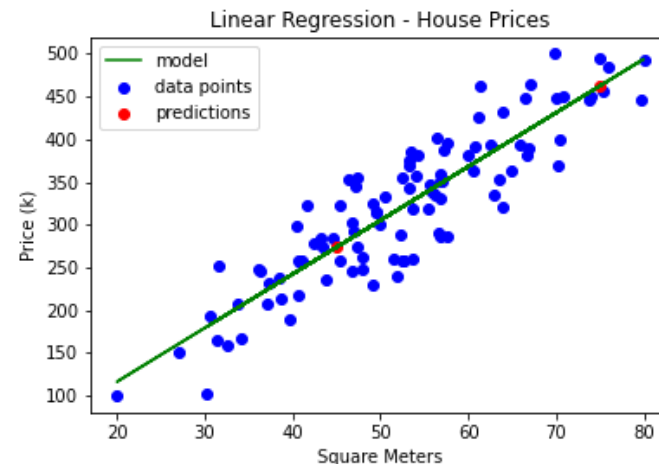
Hypothesis:

$$h_{\theta}(x) = \theta^T x = \sum_{j=0}^n \theta_j x_j = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

e.g., $h_{\theta}(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2$ in a 2-D feature space

Gradient descent:

$$\begin{aligned} \theta_j &:= \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \\ &= \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)} \end{aligned}$$



Recap – What We have Learned So Far

- Linear logistic regression

Hypothesis:

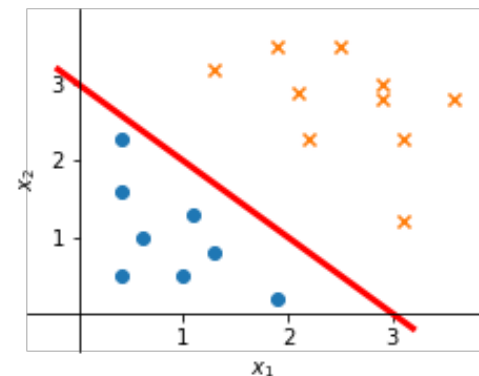
$$h_{\theta}(x) = \frac{1}{1+e^{-\theta^T x}}$$

Decision boundary:

$$\theta^T x = \sum_{j=0}^n \theta_j x_j = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = 0$$

$$\text{e.g., } \theta^T x = -3 + x_1 + x_2 = 0$$

Can you find the intercepts on x_1 and x_2 ? If so, you can plot the decision boundary.



Recap – What We have Learned So Far

- Linear SVM

Hypothesis:

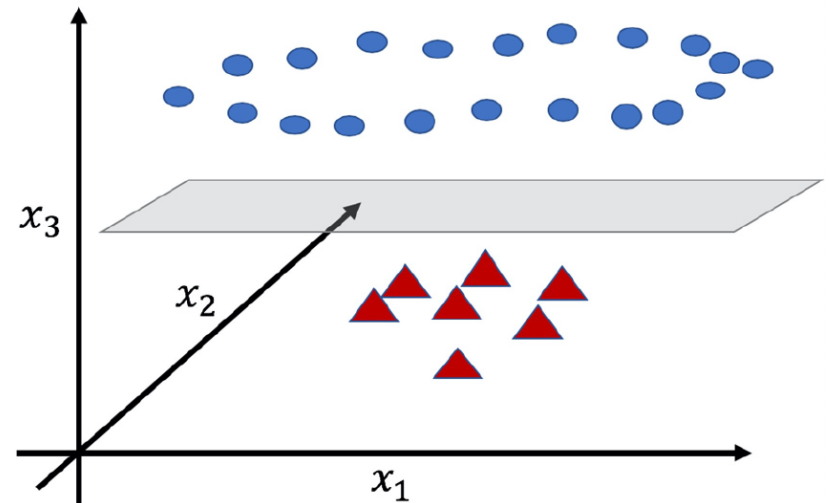
$$wx + b = 1, wx + b = 0, wx + b = -1$$

e.g.

$$w_1x_1 + w_2x_2 + w_3x_3 + b = 1$$

$$w_1x_1 + w_2x_2 + w_3x_3 + b = 0$$

$$w_1x_1 + w_2x_2 + w_3x_3 + b = -1$$



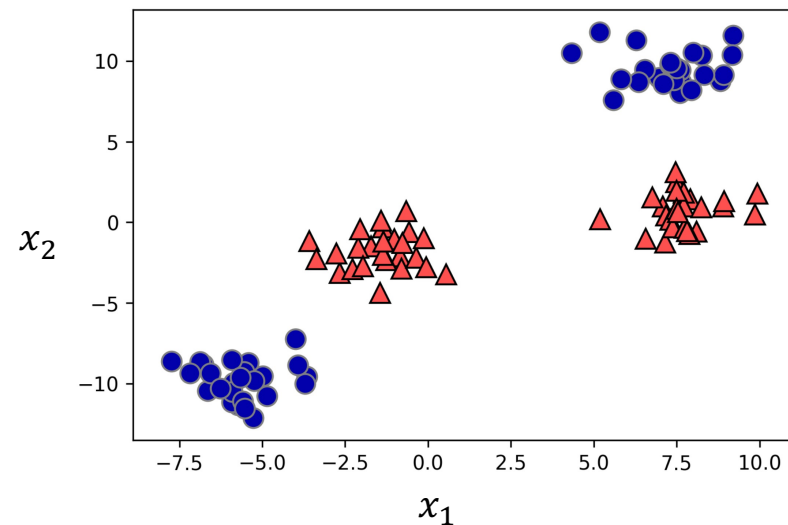
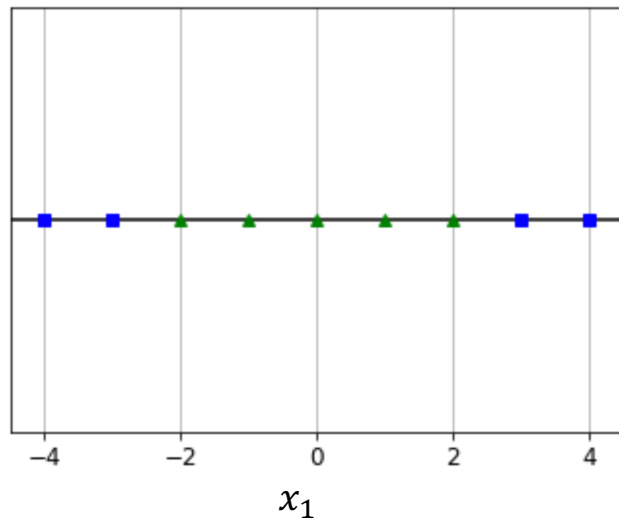
Non-linear Classification: Kernels

Kernels

- What is non-linear classification?
- What are kernels?
- Why do we need kernels?
- The polynomial kernel
- The radial basis function kernel
- How to use these kernels?

Non-linear Classification

- Linear classification – data in two classes can be separated by a *linear model*, e.g., use a straight line, a plane, a hyperplane etc.
- Non-linear classification, cannot be separated by a linear model, e.g., data can be separated only by a circle, a curved line, a bent plane etc.

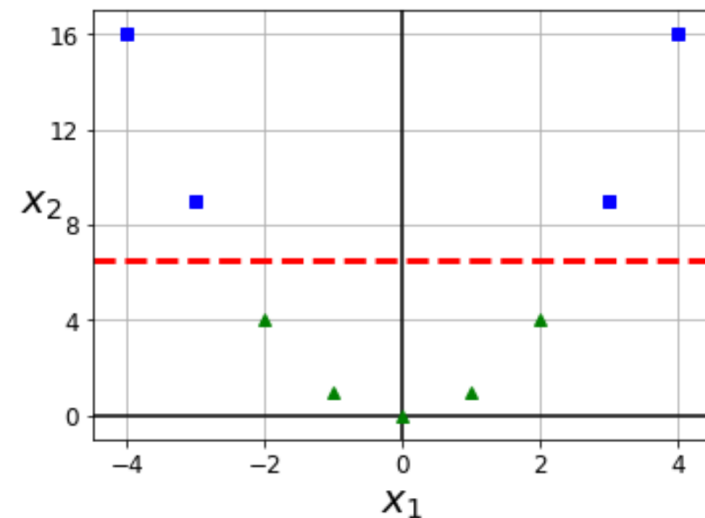
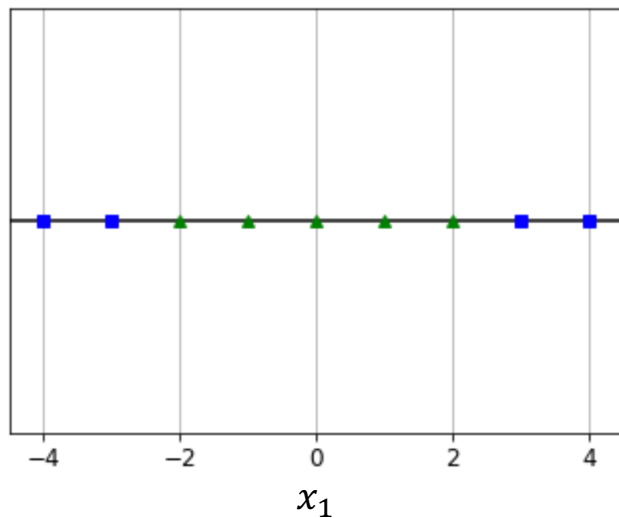


Transform data into a higher dimensional space

- We can use a *mapping function*, ϕ , to transform data in a low dimensional feature space into a higher dimensional space to linearly separate the data
For example, we can transform data in a 1-d space into a 2-d space, by adding the second dimension as follows:

$$\phi(x) = \phi([x_1]) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_1 \\ (x_1)^2 \end{bmatrix}$$

Note that we have added an additional *polynomial feature* based on the original feature.

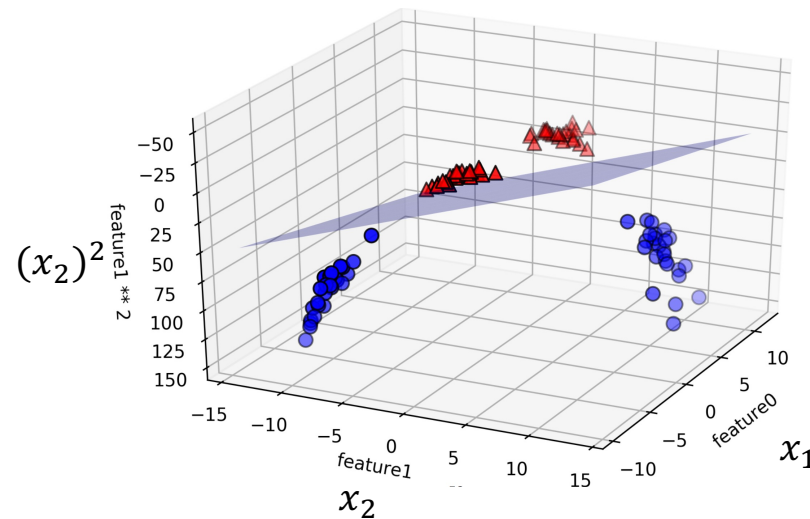
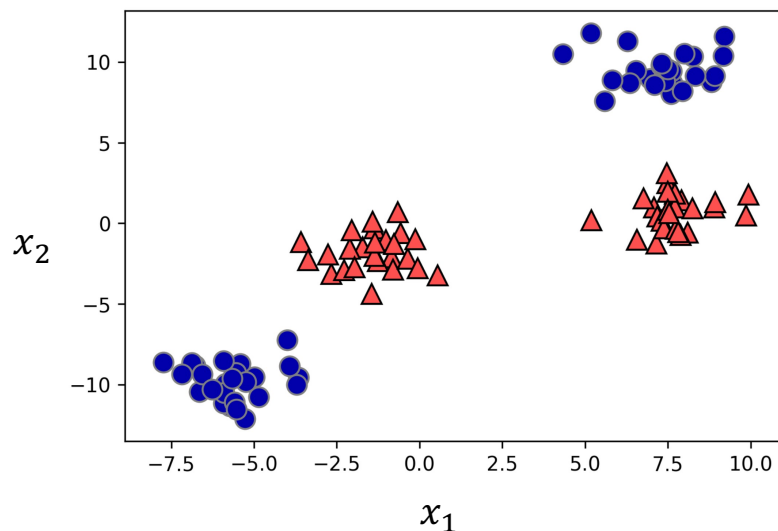


Transform data into a higher dimensional space

- We can use another mapping function, ϕ , to transform data in a low dimensional feature space into a higher dimensional space to linearly separate the data

For example, we can also transform data in a 2-d space into a 3-d space, by adding the third dimension as follows:

$$\phi(x) = \phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} x_1 \\ x_2 \\ (x_2)^2 \end{bmatrix}$$



Distance between Data Points in a Feature Space

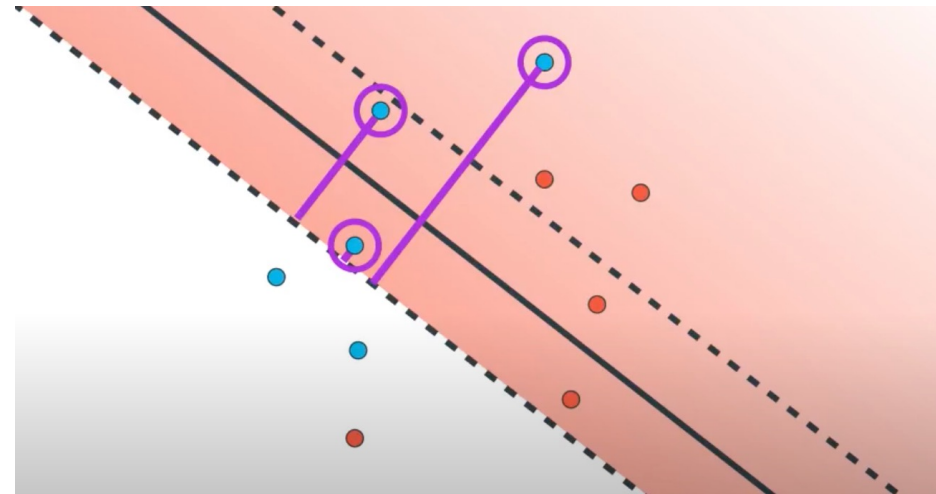
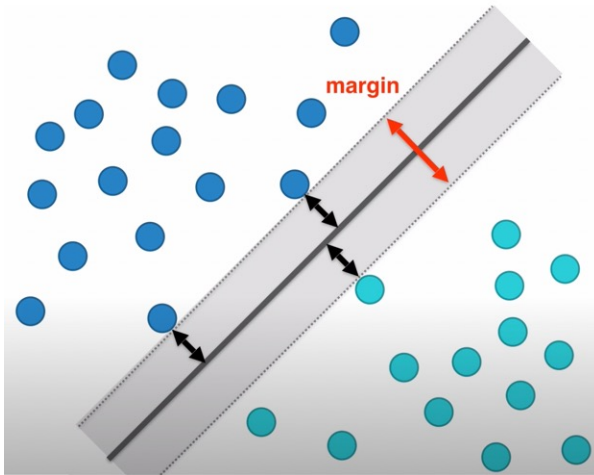
- We have so far known that we need only *the distance between every pair of data points* to find the decision boundary in an SVM
- Given a dataset of n data points in a feature space, we can use a matrix of n by n to represent the distance between every pair of data points
- The Euclidean distance between each pair of n -dimensional data points, $x^{(1)}$ and $x^{(2)}$, which are represented by two vectors, is the dot product of the vectors:

$$d(x^{(1)}, x^{(2)}) = x^{(1)} \cdot x^{(2)} = \sqrt{\sum_{i=1}^n (x_i^{(1)} - x_i^{(2)})^2}$$

Pythagorean theorem says that $d^2 = (x_1^{(1)} - x_1^{(2)})^2 + (x_2^{(1)} - x_2^{(2)})^2$.

Euclidean distance requires that the values of all features (i.e. coordinates of the data points) are normalised.

Distances between Data Points Needed



- Need to know distances between data points for finding support vectors, calculating margins, and calculating error rates
- Need to know the values of features in all dimensions, in particular, *in the transformed higher dimensions*

Kernel Functions/Tricks

- Transforming data into a high dimensional space can be *computationally extremely expensive or infeasible* when there are a large number of features, and the dimensionality of the transformation is very high or infinite.
- In reality, a *kernel function/trick* allows us to use every pair of data points in a low feature space to calculate the distance between them in a high dimensional space (i.e., inner dot product of transformed data points).
- Since we don't need to directly do the inner dot product of transformed data points, *we don't need to know explicitly the transformed data points* (i.e., their features in the higher dimensional space).
- Think of an intuitive example: Given two houses in a street, if you phone someone in the street to give you the coordinates of those two houses so you can calculate the distance between them, they will probably feel you are a bit over-complicating the problem – “Why cannot you simply ask me to give you an estimate of the distance that would be much simpler?”

Kernel Functions/Tricks

- The distance between every pair of data points in the high dimensional space alone, calculated by a *kernel function*, is sufficient to allow the maximal/soft margin and the decision boundary to be found because there are algorithms that can find the support vectors in the high dimensional space
- We can now use a kernel function to calculate each element in a *kernel matrix* of n by n , which is the dot product of two data points in a higher dimensional space
- Note that kernel functions are for *mathematical convenience* only but don't change the nature of the classification, i.e., the classification remains the same, hence called *kernel tricks*

Popular Kernels

- Linear kernels – covered already, the kernel function is the dot product itself
- Polynomial kernels
- Radial Basis Function (RBF) Kernels
- *Sigmoid Kernels and other kernels – which you may try*
- Given an application, our task is to choose a kernel and the hyper-parameters to linearly separate the data

Polynomial Kernels

- Polynomial kernel functions are defined as:

$$k(x^{(i)}, x^{(j)}) = ((x^{(i)})^T x^{(j)} + r)^d$$

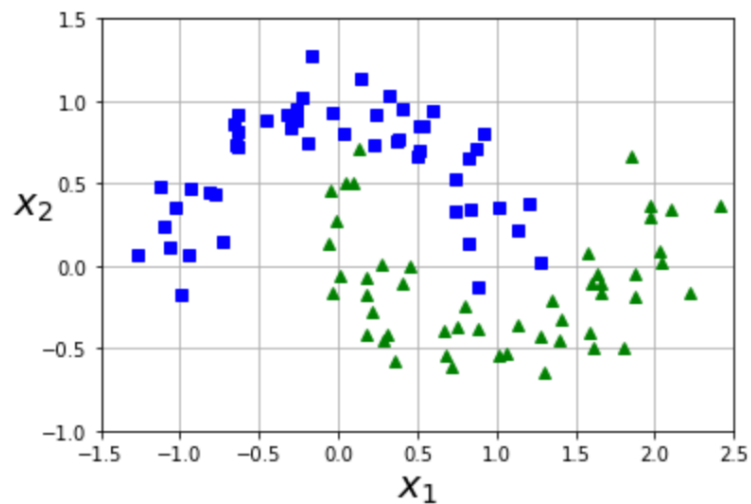
- where $x^{(i)}$ and $x^{(j)}$ are data points in a feature space, d is a hyperparameter - the degree of the polynomial, r is a hyperparameter balancing the influence of higher-order and lower-order terms in the polynomial
- For example, in a 2-d space, (x_1, x_2) , any pair of data points can be represented as $x^{(i)} = \begin{bmatrix} x_1^{(i)} & x_2^{(i)} \end{bmatrix}$ and $x^{(j)} = \begin{bmatrix} x_1^{(j)} & x_2^{(j)} \end{bmatrix}$
- We can have a polynomial kernel, with $d = 2$ and $r = 0$:

$$k(x^{(i)}, x^{(j)}) = ((x^{(i)})^T x^{(j)})^2$$

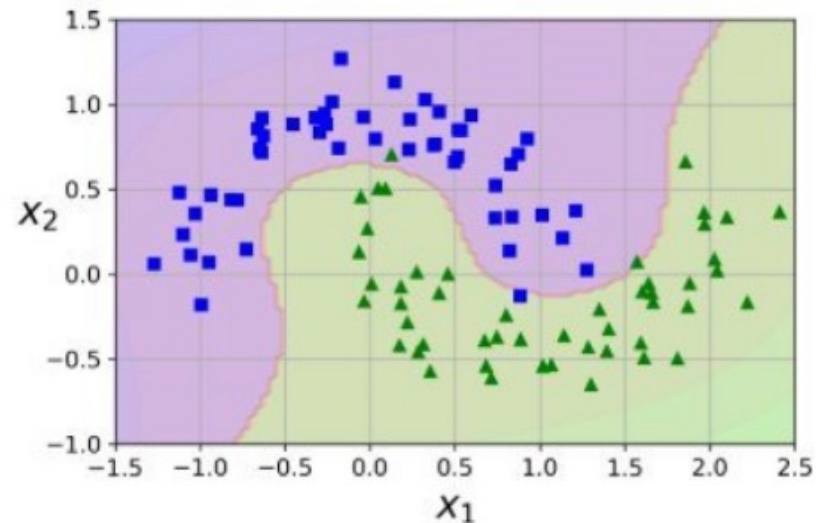
- This kernel corresponds to a particular 3-d space, $((x_1)^2, (x_2)^2, \sqrt{2}x_1x_2)$
- This kernel can be used to calculate the distance between any pair of data points in the 3-d space

Note that we don't actually generate the 3-d feature space. In the sense, r decides on the weights of features in the polynomial. Normally you would not use a value greater than 10 for d . Both r and d are positive integers.

Polynomial kernels – an Example

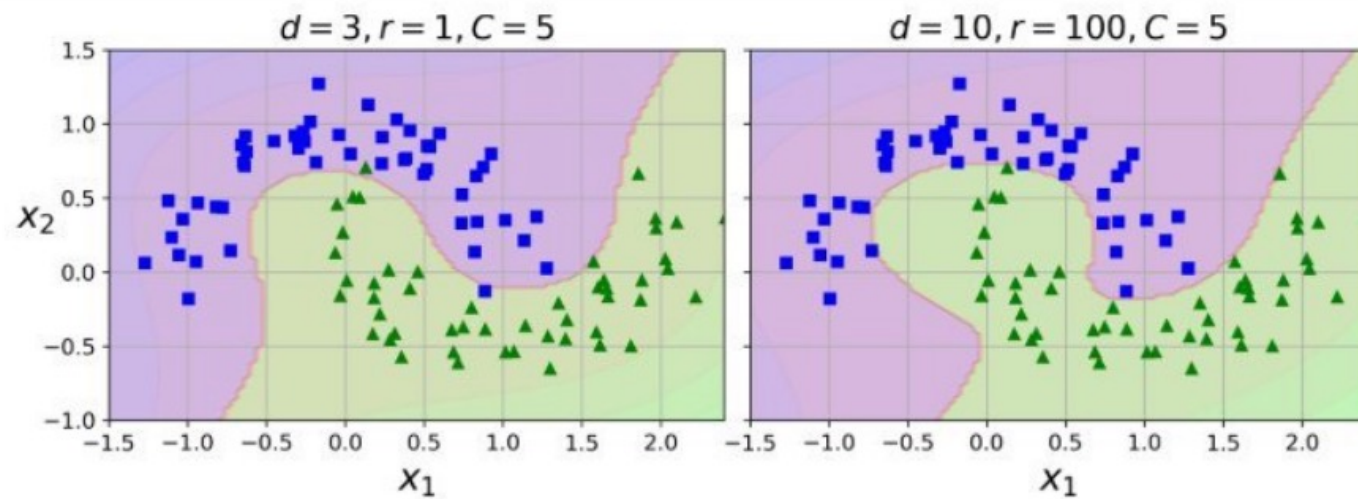


(a) Data points in the 2-d space which cannot be separated linearly



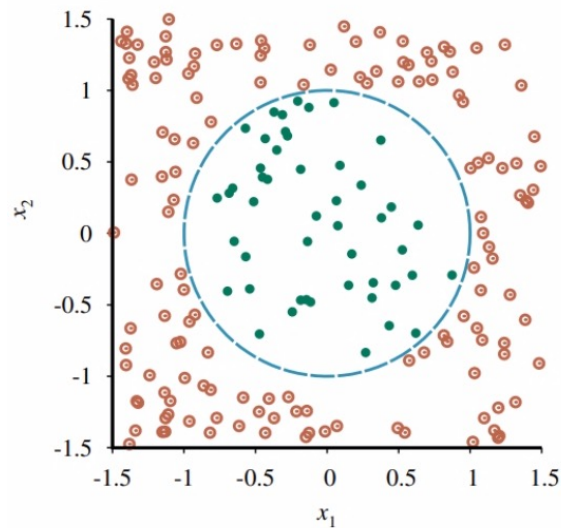
(b) Linear SVM classifier with polynomial features ($d = 3, r = 1, C = 5$) – decision boundary projected back to the 2-d space

Polynomial kernels – an Example

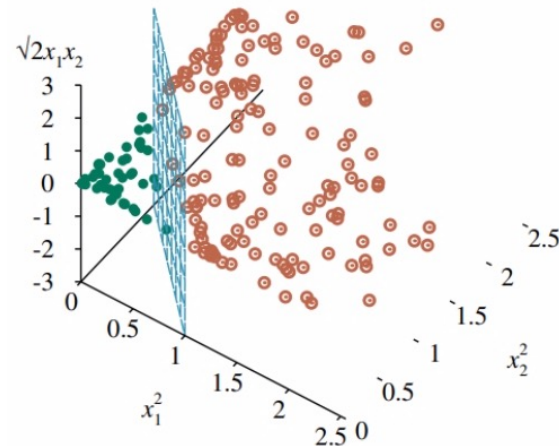


SVM classifiers with a polynomial kernel – decision boundary projected back to the 2-d space

Polynomial Kernels – Another Example



(a) Data points in the 2-d space which cannot be separated linearly



(b) Data points transformed into the 3-d space which can be separated linearly

Radial Basis Function (RBF) Kernels

- It finds a support vector classifier in an infinite dimensions
- The RBF (in sklearn) is defined as:

$$k(x^{(i)}, x^{(j)}) = e^{-\gamma \|x^{(i)} - x^{(j)}\|^2}$$

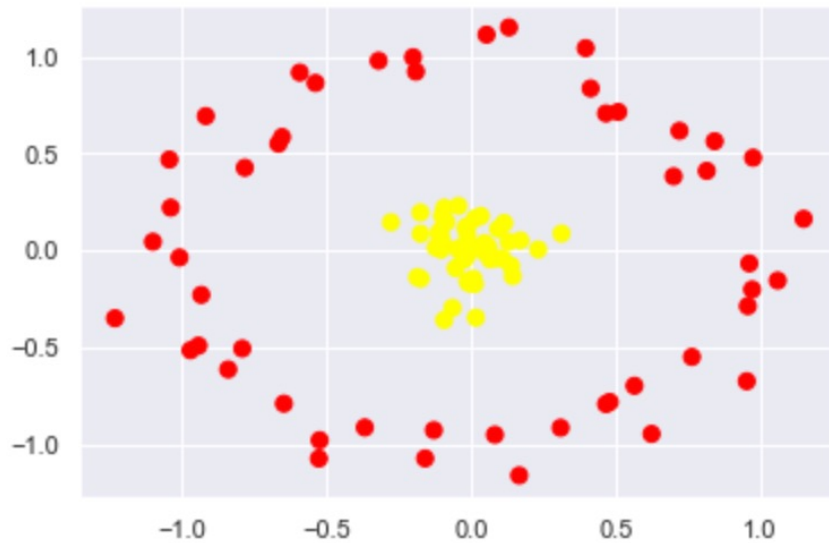
where $\|x^{(i)} - x^{(j)}\|^2$ is the squared **Euclidean distance**, $\gamma = \frac{1}{2\sigma^2}$, with $\sigma > 0$, where σ is a hyperparameter representing the radius of influence of support vectors

- It is important to choose the right value of σ which decides how the distance between a pair of data points is calculated in a higher-dimensional space
- Note that $\gamma = \frac{1}{2\sigma^2}$, i.e. γ is inversely proportional to σ .
- The RBF Kernel in the scikit-learn library actually has two hyperparameters C for the soft margin classifier and γ , and finding the right values for them is essential in order to achieve the best Bias-Variance trade off
- Refer to this section for detail:

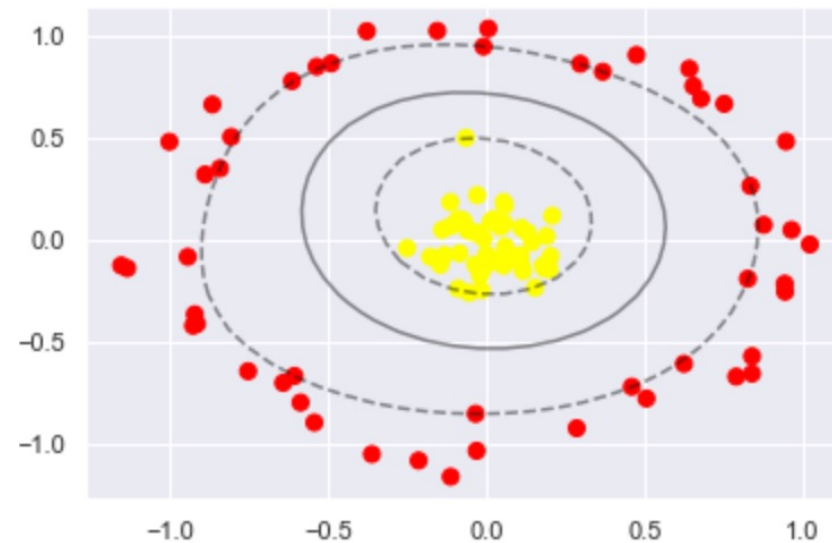
https://scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html

A low σ or high γ indicates narrower margin/overfitting (low bias and high variance) while a high σ or low γ indicates wider margin/underfitting (high bias and low variance).

Radial Basis Function (RBF) Kernels – an Example



(a) Data points in the 2-d space which cannot be separated linearly



(b) SVM classifiers with a RBF kernel – decision boundary projected back to the 2-d space

SVM – Pros and Cons

- Advantages
 - Linear and non-linear classification
 - Require small/medium datasets only (as we need support vectors only)
 - Good for high-dimensional data – we simply choose a kernel function and the corresponding higher-dimensional feature space (we don't even need to know or be able to visualise the space) to linearly separate the data points in the higher-dimensional space
- Disadvantages
 - Computationally expensive hence not good for large datasets
 - Needs a lot of experience and experiments in choosing hyperparameters

References

- Chapter 5. Support Vector Machines in Hands-on ML

Resources

- In-Depth: Support Vector Machines
 - <https://jakevdp.github.io/PythonDataScienceHandbook/05.07-support-vector-machines.html>
 - Excerpt from *[Python Data Science Handbook](#)*
 - Good coverage of IPython, NumPy, Pandas, Matplotlib, Seaborn, feature engineering, linear regression, SVM, random forests
- There are a lot of other online resources
 - Search for them if interested